

PROYECTO DE MACHINE LEARNING

Implementación, Evaluación y Optimización de Modelos para la Predicción del Rendimiento Académico

Documento de desarrollo, evidencias y resultados

Institución	Universidad Tecnológica
Asignatura	Extracción de BD
Docente	Eliseo Izquierdo
Integrantes	David, Devany, Alexis y Ángel
Grupo	IDGS9
Fecha de entrega	29/07/2026

Índice de contenido

Contenido

Índice de contenido.....	2
Introducción	6
Caso de estudio	6
Objetivo general	6
Objetivos específicos	6
Organización inicial del proyecto	6
David — Coordinación y modelo de clasificación	6
Devany — Dataset y preprocesamiento	7
Alexis — Modelo de regresión	7
Ángel — Visualización y documentación	7
Variables del dataset	7
Algoritmos candidatos	7
Herramientas de trabajo	8
Estructura del repositorio	8
Ramas de trabajo.....	8
Fase 1. Construcción del dataset.....	9
Objetivo de la fase	9
Procedimiento.....	9
1. Definición de rangos	9
2. Creación de registros.....	10
3. Validación inicial	11
4. Exportación	11
Resultados obtenidos	12
Problemas encontrados y solución aplicada.....	14
Conclusión de la fase	14
Fase 2. Análisis del problema	15
Objetivo de la fase	15
Procedimiento.....	15
1. Problema de clasificación.....	15
2. Problema de regresión	15

3. Variables de entrada.....	16
4. Prevención de fuga de información	17
Resultados obtenidos	18
Problemas encontrados y solución aplicada.....	22
Conclusión de la fase	22
Fase 3. Justificación de los algoritmos	23
Objetivo de la fase	23
Procedimiento.....	23
1. Clasificación	23
2. Regresión	24
3. Criterios de comparación	26
4. Modelo base.....	28
Resultados obtenidos	28
Problemas encontrados y solución aplicada.....	30
Conclusión de la fase	30
Fase 4. Implementación de los modelos	30
Objetivo de la fase	30
Desarrollar los notebooks de clasificación y regresión en Google Colab, preparando los datos, entrenando los algoritmos candidatos y generando predicciones con el conjunto de prueba.	30
Procedimiento.....	30
1. Importación de librerías.....	30
2. Lectura del dataset.....	31
3. Preparación.....	32
4. División de datos	33
5. Entrenamiento	34
6. Predicción.....	35
Resultados obtenidos	37
Evidencias opcionales	37
Problemas encontrados y solución aplicada.....	40
Conclusión de la fase	41
Fase 5. Evaluación	41
Objetivo de la fase	41
Procedimiento.....	41
1. Clasificación	41

2. Regresión	42
3. Interpretación	42
4. Comparación	43
Resultados obtenidos	44
Evidencia 2. Matriz de confusión del mejor modelo de clasificación	45
Evidencia 3. Comparación de métricas de regresión	46
Problemas encontrados y solución aplicada.....	46
Conclusión de la fase	46
Fase 6. Optimización.....	47
Objetivo de la fase	47
Aplicar cambios en los modelos de clasificación y regresión para intentar mejorar su desempeño, comparar los resultados antes y después de la optimización y revisar si existe sobreajuste.	47
Procedimiento.....	47
1. Selección de estrategia.....	47
2. Reentrenamiento.....	47
3. Nueva evaluación	47
4. Sobreajuste.....	47
5. Conclusión de mejora.....	47
Resultados obtenidos	48
Problemas encontrados y solución aplicada.....	49
Conclusión de la fase	49
Fase 7. Resultados	50
Objetivo de la fase	50
Seleccionar los mejores modelos, analizar las variables que tuvieron mayor influencia en las predicciones y explicar cómo podrían utilizarse los resultados para detectar estudiantes en riesgo académico.	50
Procedimiento.....	50
1. Mejor algoritmo	50
2. Razones	50
3. Variables importantes	51
4. Aprendizajes	51
5. Aplicación	52
Resultados obtenidos	52
Problemas encontrados y solución aplicada.....	54

Conclusión de la fase	54
Fase 8. Publicación en GitHub	54
Objetivo de la fase	54
Organizar y publicar el proyecto con todos los archivos solicitados, verificando que el repositorio tenga una estructura clara, que los notebooks puedan ejecutarse correctamente y que la documentación sea accesible.	54
Procedimiento.....	54
1. Carga de archivos.....	54
2. README.....	55
3. Revisión de ejecución	56
4. Verificación final.....	56
Resultados obtenidos	57
Problemas encontrados y solución aplicada.....	58
Conclusión de la fase	58
Conclusiones generales	58
Bibliografía.....	59

Introducción

El rendimiento académico puede analizarse mediante información relacionada con el desempeño y los hábitos de los estudiantes. En este proyecto se desarrollarán dos modelos de Machine Learning supervisado: uno de clasificación para estimar si un alumno aprobará o reprobará, y otro de regresión para predecir su calificación final.

El desarrollo incluirá la construcción del conjunto de datos, la preparación de las variables, el entrenamiento, la evaluación, la optimización y la publicación de los resultados. Este documento registrará cada procedimiento realizado y las evidencias obtenidas durante el proyecto.

Caso de estudio

La Universidad Tecnológica desea contar con un sistema inteligente que ayude a los coordinadores académicos a tomar decisiones antes de que los estudiantes reprobren. Para ello, se utilizarán datos como el promedio parcial, la asistencia, las horas de estudio, los accesos a Moodle y las tareas entregadas.

El sistema deberá responder dos preguntas: si un estudiante aprobará o reprobará y cuál será su calificación final.

Objetivo general

Desarrollar un proyecto completo de Machine Learning supervisado mediante la implementación de un modelo de clasificación y un modelo de regresión, evaluando su desempeño con métricas especializadas y aplicando procesos de optimización para mejorar los resultados.

Objetivos específicos

- Construir un dataset de aproximadamente 200 registros con las variables establecidas.
- Preparar y analizar los datos antes de entrenar los modelos.
- Implementar y comparar diferentes algoritmos de clasificación y regresión.
- Evaluar los modelos mediante las métricas solicitadas.
- Optimizar los modelos y comparar el desempeño antes y después de los cambios.
- Documentar el proceso y publicar los archivos en un repositorio de GitHub.

Organización inicial del proyecto

Cada integrante tendrá una responsabilidad principal, pero todos participarán en la revisión y deberán conocer el funcionamiento general del proyecto.

David — Coordinación y modelo de clasificación

- Organizar el repositorio de GitHub.
- Crear la estructura inicial del proyecto.
- Coordinar la integración de los archivos.

- Desarrollar y evaluar el modelo de clasificación.
- Comparar los algoritmos de clasificación.
- Revisar el código final antes de publicarlo.

Devany — Dataset y preprocesamiento

- Crear aproximadamente 200 registros en Excel.
- Verificar la coherencia de las variables.
- Detectar datos faltantes, duplicados o incorrectos.
- Limpiar los datos con Pandas.
- Preparar la variable Resultado para el modelo.
- Documentar el significado de cada columna.

Alexis — Modelo de regresión

- Desarrollar el modelo para predecir la calificación final.
- Entrenar los algoritmos de regresión.
- Calcular e interpretar las métricas.
- Comparar valores reales y predichos.
- Analizar los errores.
- Seleccionar el modelo con mejor desempeño.

Ángel — Visualización y documentación

- Crear gráficas del análisis de datos.
- Elaborar matrices de confusión.
- Crear gráficas de valores reales contra predichos.
- Organizar capturas y evidencias.
- Apoyar en resultados y conclusiones.
- Integrar el reporte final.

Variables del dataset

Campo	Nombre sugerido en CSV	Descripción
Promedio parcial	Promedio_Parcial	Promedio del estudiante antes de la calificación final.
Asistencia	Asistencia	Porcentaje de asistencia.
Horas estudio	Horas_Estudio	Cantidad de horas dedicadas al estudio en el periodo definido.
Accesos Moodle	Accesos_Moodle	Número de accesos a la plataforma Moodle.
Tareas	Tareas	Número de tareas entregadas.
Calificación final	Calificacion_Final	Calificación numérica final, de 0 a 100.
Resultado	Resultado	Categoría Aprueba o Reprueba.

Algoritmos candidatos

Clasificación: Regresión Logística, Árbol de Decisión y Random Forest.

Regresión: Regresión Lineal, Árbol de Decisión para regresión y Random Forest Regressor.

La selección final se realizará mediante la comparación de métricas y el análisis de estabilidad de los modelos.

Herramientas de trabajo

- Microsoft Excel
- Google Colab
- Python
- Pandas
- NumPy
- Scikit-Learn
- Matplotlib
- GitHub
- Draw.io (opcional)

Estructura del repositorio

```
Proyecto_MachineLearning/  
├── Dataset/  
│   └── alumnos.csv  
├── Regresion/  
│   └── regresion.ipynb  
├── Clasificacion/  
│   └── clasificacion.ipynb  
├── Imagenes/  
│   └── graficas.png  
├── Documentacion/  
│   └── Reporte.pdf  
└── README.md
```

Ramas de trabajo

- main: versión estable.
- david-clasificacion: modelo de clasificación.
- devany-dataset: dataset y limpieza.
- alexis-regresion: modelo de regresión.
- angel-documentacion: gráficas y reporte.

Fase 1. Construcción del dataset

Objetivo de la fase

Crear el archivo alumnos.csv con aproximadamente 200 registros y los siete campos establecidos:

- Promedio parcial.
- Asistencia.
- Horas de estudio.
- Accesos a Moodle.
- Tareas.
- Calificación final.
- Resultado.

Esta fase corresponde a la construcción del conjunto de datos solicitado para entrenar posteriormente los modelos de clasificación y regresión.

Procedimiento

1. Definición de rangos

Establecer las escalas y límites válidos de cada variable antes de crear los registros.

Procedimiento realizado: Antes de generar los registros, se definieron los rangos permitidos para cada variable. Esto se realizó con el propósito de mantener uniformidad en los datos, evitar valores imposibles y facilitar la validación posterior del archivo.

Los rangos establecidos fueron los siguientes:

Variable	Tipo de dato	Rango o valores permitidos
<i>Promedio parcial</i>	<i>Numérico entero</i>	<i>0 a 100</i>
<i>Asistencia</i>	<i>Numérico entero</i>	<i>0 a 100</i>
<i>Horas de estudio</i>	<i>Numérico</i>	<i>0 a 20</i>
<i>Accesos a Moodle</i>	<i>Numérico entero</i>	<i>0 a 60</i>
<i>Tareas</i>	<i>Numérico entero</i>	<i>0 a 10</i>
<i>Calificación final</i>	<i>Numérico entero</i>	<i>0 a 100</i>
<i>Resultado</i>	<i>Categorico</i>	<i>Aprueba o Reprueba</i>

La variable de asistencia representa un porcentaje, mientras que las horas de estudio corresponden a las horas semanales dedicadas por el estudiante. Se estableció un máximo de diez tareas y un máximo de sesenta accesos a Moodle durante el periodo analizado.

Para determinar el resultado académico se definió como calificación mínima aprobatoria un valor de 70 puntos:

Calificacion_Final $\geq$ 70 $\rightarrow$ Aprueba

Calificacion_Final $<$ 70 $\rightarrow$ Reprueba

2. Creación de registros

Generar los datos en Excel manteniendo relaciones coherentes entre las variables.

Procedimiento realizado: Se creó una hoja de cálculo en Microsoft Excel con las siguientes columnas:

- Promedio_Parcial
- Asistencia
- Horas_Estudio
- Accesos_Moodle
- Tareas
- Calificacion_Final
- Resultado

Posteriormente, se generaron aproximadamente 200 registros de estudiantes. Durante este proceso se procuró que las variables mantuvieran relaciones académicas coherentes.

En términos generales, los estudiantes con mayor promedio parcial, asistencia, horas de estudio, accesos a Moodle y tareas entregadas recibieron calificaciones finales más altas. Sin embargo, también se incorporaron variaciones para evitar que los datos fueran completamente predecibles.

Por ejemplo, se incluyeron casos como:

- Estudiantes con buena asistencia, pero pocas tareas entregadas.
- Estudiantes con varias horas de estudio y promedio parcial bajo.
- Estudiantes con muchos accesos a Moodle, pero rendimiento medio.
- Estudiantes que mejoraron su desempeño en la calificación final.
- Estudiantes cuyo rendimiento disminuyó respecto al promedio parcial.

La calificación final se asignó considerando el comportamiento conjunto de las variables, evitando que fuera una copia exacta del promedio parcial. Finalmente, la columna Resultado se determinó según la calificación final obtenida.

Ejemplo de la estructura utilizada:

<i>Promedio_Parcial</i>	<i>Asistencia</i>	<i>Horas_Estudio</i>	<i>Accesos_Moodle</i>	<i>Tareas</i>	<i>Calificacion_Final</i>	<i>Resultado</i>
85	92	8	42	9	88	Aprueba
62	68	3	15	5	64	Reprueba
74	82	6	31	8	77	Aprueba
55	60	2	9	3	52	Reprueba
91	97	12	54	10	94	Aprueba

3. Validación inicial

Revisar datos fuera de rango, duplicados, valores vacíos y contradicciones.

Procedimiento realizado: Se verificaron los siguientes aspectos:

- Que el promedio parcial estuviera entre 0 y 100.
- Que la asistencia estuviera entre 0 y 100.
- Que las horas de estudio estuvieran entre 0 y 20.
- Que los accesos a Moodle fueran números enteros entre 0 y 60.
- Que las tareas estuvieran entre 0 y 10.
- Que la calificación final estuviera entre 0 y 100.
- Que no existieran celdas vacías.
- Que no existieran filas duplicadas.
- Que las columnas numéricas no contuvieran texto.
- Que los valores de resultado fueran únicamente Aprueba o Reprueba.
- Que el resultado correspondiera con la calificación final.

También se revisaron posibles contradicciones. Por ejemplo, no se permitió que un estudiante con una calificación final menor a 70 apareciera como aprobado, ni que un estudiante con una calificación igual o superior a 70 apareciera como reprobado.

Durante la revisión se corrigieron los registros que presentaban valores fuera de rango, errores de escritura o inconsistencias entre la calificación final y el resultado.

4. Exportación

Guardar el archivo final con el nombre alumnos.csv en formato CSV.

Procedimiento realizado: Después de completar y validar los registros, el archivo fue exportado desde Microsoft Excel utilizando el formato: CSV UTF-8 delimitado por comas. El archivo final se guardó con el nombre: *alumnos.csv*

Posteriormente, se colocó dentro de la carpeta correspondiente del repositorio:

`Proyecto_MachineLearning/`

└── `Dataset/`

└── `alumnos.csv`

Finalmente, se abrió nuevamente el archivo para comprobar que los encabezados, las filas y los caracteres se hubieran guardado correctamente.

Resultados obtenidos

Como resultado de esta fase se obtuvo un conjunto de datos con aproximadamente 200 registros y siete variables relacionadas con el rendimiento académico de los estudiantes.

El archivo contiene información sobre el promedio parcial, asistencia, horas de estudio, accesos a Moodle, tareas entregadas, calificación final y resultado académico.

Los registros fueron diseñados para mantener relaciones coherentes entre las variables, aunque también se incluyeron variaciones que representan diferentes comportamientos estudiantiles. Esto permitirá que los modelos identifiquen patrones sin depender completamente de una sola característica.

La validación inicial permitió confirmar que:

- Las variables se encuentran dentro de los rangos definidos.
- No existen registros duplicados.
- No existen valores vacíos.
- Los datos numéricos mantienen el formato correcto.
- La columna Resultado coincide con la calificación final.
- El archivo puede utilizarse posteriormente en Google Colab.

El producto final de esta fase fue el archivo: `alumnos.csv`

Este archivo será utilizado en las siguientes fases para analizar el problema, preparar los datos y entrenar los modelos de Machine Learning.

Evidencias de la Fase 1

Evidencia 1. Tabla de rangos de las variables

Nombre de la variable	Tipo de dato	Valor mínimo	Valor máximo	Descripción
Promedio_Parcial	Numérico entero	0	100	Promedio académico obtenido por el estudiante antes de la evaluación final.
Asistencia	Numérico entero	0	100	Porcentaje de clases a las que asistió el estudiante durante el periodo evaluado.
Horas_Estudio	Numérico decimal	0	20	Cantidad promedio de horas semanales que el estudiante dedica al estudio fuera de clase.
Accesos_Moodle	Numérico entero	0	60	Número de veces que el estudiante ingresó a la plataforma Moodle durante el periodo evaluado.
Tareas	Numérico entero	0	10	Cantidad de tareas entregadas por el estudiante, considerando un total de diez actividades.
Calificacion_Final	Numérico entero	0	100	Calificación obtenida por el estudiante al finalizar el periodo académico.
Resultado	Categorico	Reprueba	Aprueba	Resultado académico determinado a partir de la calificación final. Una calificación igual o superior a 70 corresponde a “Aprueba” y una menor a 70 corresponde a “Reprueba”.

Evidencia 2. Captura del archivo en Excel

Dataset de rendimiento académico — 200 estudiantes						
Promedio_Parcial	Asistencia	Horas_Estudio	Accesos_Moodle	Tareas	Calificacion_Final	Resultado
92	81	6.2	40	9	90	Aprueba
56	100	12.4	30	7	78	Aprueba
57	78	3.4	23	7	67	Reprueba
52	83	6.4	35	7	73	Aprueba
87	85	9.3	33	10	96	Aprueba
71	83	10.7	34	9	84	Aprueba
52	62	3.1	17	2	54	Reprueba
72	76	8.8	36	9	77	Aprueba
93	89	14.9	41	8	95	Aprueba
76	71	8.7	22	6	71	Aprueba
76	100	11.1	45	10	99	Aprueba
75	99	11.2	45	8	87	Aprueba
80	100	8.1	49	10	93	Aprueba
56	84	4.7	29	8	71	Aprueba
72	94	7.4	45	10	81	Aprueba
57	85	9.2	31	10	81	Aprueba
51	75	0.3	0	0	46	Reprueba
60	72	1.7	39	7	76	Aprueba
31	47	0.0	11	3	35	Reprueba
50	87	4.5	12	3	69	Reprueba
72	93	9.5	36	7	84	Aprueba
68	88	10.2	39	8	88	Aprueba
34	67	2.7	14	5	47	Reprueba
68	100	9.2	31	9	91	Aprueba
61	84	5.6	27	7	53	Reprueba
77	80	8.5	36	10	79	Aprueba
63	64	2.6	23	7	69	Reprueba
41	68	1.6	6	1	47	Reprueba
80	87	9.5	29	7	85	Aprueba
100	100	12.0	55	10	100	Aprueba
77	97	10.6	48	7	88	Aprueba
80	86	11.8	45	10	94	Aprueba
73	91	8.4	41	7	83	Aprueba
85	87	12.0	46	10	87	Aprueba
99	100	14.6	37	10	100	Aprueba
47	63	4.3	0	7	61	Reprueba
37	81	0.0	26	5	52	Reprueba
79	78	7.1	29	7	82	Aprueba
76	80	7.2	35	9	85	Aprueba
55	79	2.9	18	7	62	Reprueba
43	65	4.4	13	4	57	Reprueba
62	72	7.8	24	7	69	Reprueba
100	89	13.6	48	10	100	Aprueba
37	64	0.0	4	4	52	Reprueba

Evidencia 3. Validación del Dataset

Validación inicial del dataset					
Variable	Tipo	Mínimo permitido	Máximo permitido	Mínimo obtenido	Máximo obtenido
Promedio_Parcial	Entero	0	100	23	100
Asistencia	Entero	0	100	38	100
Horas_Estudio	Decimal	0	20	0	16.1
Accesos_Moodle	Entero	0	60	0	60
Tareas	Entero	0	10	0	10
Calificacion_Final	Entero	0	100	15	100
Resultado	Categorico	Reprueba	Aprueba		
Indicador		Resultado			
Total de registros	200	Criterio de resultado: Calificación final $\geq$ 70: Aprueba Calificación final $<$ 70: Reprueba			
Total de campos	7				
Valores vacíos	0	La distribución fue controlada para obtener 120 aprobados y 80			
Registros duplicados	0				
Registros fuera de rango	0				
Contradicciones resultado/califica	0				
Estudiantes aprobados	120				
Estudiantes reprobados	80				
Porcentaje de aprobados	60%				
Porcentaje de reprobados	40%				

Problemas encontrados y solución aplicada

Problema: Durante la generación del dataset existía el riesgo de crear registros demasiado aleatorios, con valores fuera de los límites establecidos o con contradicciones entre la calificación final y el resultado académico. También podía generarse una cantidad desproporcionada de estudiantes aprobados o reprobados, lo que afectaría el entrenamiento del modelo de clasificación.

Solución: Se establecieron previamente los rangos válidos para cada variable y se generaron los datos manteniendo relaciones coherentes entre el promedio parcial, la asistencia, las horas de estudio, los accesos a Moodle, las tareas y la calificación final. Después, se realizó una validación para detectar valores vacíos, duplicados, datos fuera de rango y contradicciones. También se controló la distribución de la variable Resultado para mantener una proporción adecuada entre estudiantes aprobados y reprobados.

Resultado: Se obtuvo un dataset con 200 registros completos, sin valores vacíos, sin duplicados, sin datos fuera de rango y sin contradicciones entre la calificación final y el resultado. La distribución final fue de 120 estudiantes aprobados y 80 reprobados, equivalente a 60 % y 40 %, respectivamente.

Conclusión de la fase

La construcción del dataset permitió obtener una base de datos organizada, coherente y preparada para las siguientes etapas del proyecto. Los siete campos establecidos fueron incluidos correctamente y cada variable se mantuvo dentro de los rangos definidos.

La validación inicial confirmó que el archivo cumple con los requisitos necesarios para ser utilizado en Google Colab y en los modelos de Machine Learning. Además, la distribución equilibrada de los resultados ayudará a evitar que el modelo de clasificación dependa demasiado de una sola categoría.

Como producto final de esta fase se generó el archivo alumnos.csv, el cual será utilizado para analizar el problema, preparar las variables y entrenar posteriormente los modelos de clasificación y regresión.

Fase 2. Análisis del problema

Objetivo de la fase

Definir claramente qué se desea predecir en los modelos de clasificación y regresión, así como identificar las variables de entrada y las variables objetivo que se utilizarán en cada caso.

Procedimiento

1. Problema de clasificación

Predecir Resultado: Aprueba o Reprueba.

Procedimiento realizado: Se definió un problema de clasificación supervisada cuyo objetivo será predecir el resultado académico de cada estudiante. La variable que se intentará predecir será Resultado, la cual contiene dos categorías posibles:

- Aprueba
- Reprueba

El modelo recibirá información relacionada con el desempeño y la actividad académica del estudiante. A partir de estas características, deberá determinar a cuál de las dos categorías pertenece.

Durante el preprocesamiento, la variable podrá transformarse a valores numéricos para facilitar su uso en los algoritmos:

- Aprueba = 1
- Reprueba = 0

El problema corresponde a una clasificación binaria porque la salida solo puede pertenecer a una de dos clases. El modelo no intentará calcular una cantidad numérica, sino seleccionar una categoría.

La pregunta que responderá este modelo será:

¿El estudiante aprobará o reprobará?

2. Problema de regresión

Predecir Calificacion_Final en una escala de 0 a 100.

Procedimiento realizado: Se definió un problema de regresión supervisada cuyo objetivo será estimar la calificación final de cada estudiante.

La variable objetivo será:

Calificacion_Final

Esta variable contiene valores numéricos comprendidos entre 0 y 100. El modelo utilizará las características académicas del estudiante para producir una predicción numérica.

Por ejemplo, el modelo podrá generar resultados como:

68

74

82

91

El problema corresponde a regresión porque la salida esperada no es una categoría, sino un valor numérico dentro de un rango continuo.

La pregunta que responderá este modelo será:

¿Cuál será la calificación final del estudiante?

Aunque en el dataset las calificaciones se registraron como números enteros, el modelo podría producir predicciones decimales, como 76.4 o 88.7. Estos valores podrán redondearse únicamente para su presentación, pero las métricas deberán calcularse utilizando las predicciones originales.

3. Variables de entrada

Promedio_Parcial, Asistencia, Horas_Estudio, Accesos_Moodle y Tareas.

Procedimiento realizado: Se analizaron las siete columnas del dataset para determinar cuáles funcionarían como variables de entrada y cuáles serían utilizadas como objetivos.

Para ambos modelos se seleccionaron las siguientes entradas:

- Promedio_Parcial
- Asistencia
- Horas_Estudio
- Accesos_Moodle
- Tareas

Estas variables fueron seleccionadas porque contienen información disponible antes de conocer el resultado final del estudiante y representan diferentes aspectos de su desempeño académico.

Variable de entrada	Información que representa
Promedio_Parcial	Desempeño académico acumulado antes de la evaluación final.
Asistencia	Porcentaje de clases a las que asistió el estudiante.

Horas_Estudio	Tiempo semanal dedicado al estudio fuera de clase.
Accesos_Moodle	Nivel de actividad del estudiante en la plataforma educativa.
Tareas	Cantidad de tareas entregadas durante el periodo.

Las mismas cinco variables se utilizarán en los dos modelos para poder estudiar cómo influyen tanto en la probabilidad de aprobar como en la calificación final.

La separación general será:

Variables de entrada (X):

- Promedio_Parcial
- Asistencia
- Horas_Estudio
- Accesos_Moodle
- Tareas

En clasificación:

Variable objetivo (y):

- Resultado

En regresión:

Variable objetivo (y):

- Calificacion_Final

4. Prevención de fuga de información

No utilizar Calificacion_Final para predecir Resultado ni Resultado para predecir Calificacion_Final.

Procedimiento realizado: Se revisó la relación existente entre las variables Calificacion_Final y Resultado. En el dataset, la variable Resultado se determina mediante la siguiente regla:

- *Calificacion_Final $\geq 70 \rightarrow$ Aprueba*
- *Calificacion_Final $< 70 \rightarrow$ Reprueba*

Debido a esta relación directa, utilizar Calificacion_Final como entrada del modelo de clasificación provocaría una fuga de información. El algoritmo recibiría prácticamente la respuesta que debe predecir y podría obtener métricas demasiado altas sin aprender realmente de las características académicas del estudiante.

Por esta razón, el modelo de clasificación utilizará:

Entradas:

- *Promedio_Parcial*
- *Asistencia*
- *Horas_Estudio*
- *Accesos_Moodle*
- *Tareas*

Objetivo:

- *Resultado*

De manera similar, utilizar Resultado como entrada del modelo de regresión también produciría una fuga de información. Al conocer si el estudiante aprobó o reprobó, el modelo recibiría información directamente relacionada con la calificación final.

Por lo tanto, el modelo de regresión utilizará:

Entradas:

- *Promedio_Parcial*
- *Asistencia*
- *Horas_Estudio*
- *Accesos_Moodle*
- *Tareas*

Objetivo:

- *Calificacion_Final*

Esta decisión permitirá que ambos modelos aprendan únicamente a partir de las características disponibles antes de conocer el desempeño final del estudiante.

Resultados obtenidos

Como resultado de esta fase se definieron dos problemas de Machine Learning supervisado.

El primer problema corresponde a una clasificación binaria. Su objetivo será predecir si un estudiante aprobará o reprobará mediante la variable Resultado.

El segundo problema corresponde a una regresión. Su objetivo será estimar la Calificacion_Final del estudiante en una escala de 0 a 100.

Para ambos modelos se utilizarán las mismas cinco variables de entrada:

- Promedio parcial.
- Asistencia.
- Horas de estudio.
- Accesos a Moodle.
- Tareas.

También se estableció que la calificación final no será utilizada para predecir el resultado y que el resultado no será utilizado para predecir la calificación final. Esta decisión evitará la fuga de información y permitirá obtener evaluaciones más realistas.

La estructura definitiva quedó de la siguiente manera:

Modelo	Tipo de problema	Variables de entrada	Variable objetivo
Clasificación	Clasificación binaria	Promedio parcial, asistencia, horas de estudio, accesos a Moodle y tareas	Resultado
Regresión	Predicción numérica	Promedio parcial, asistencia, horas de estudio, accesos a Moodle y tareas	Calificación final

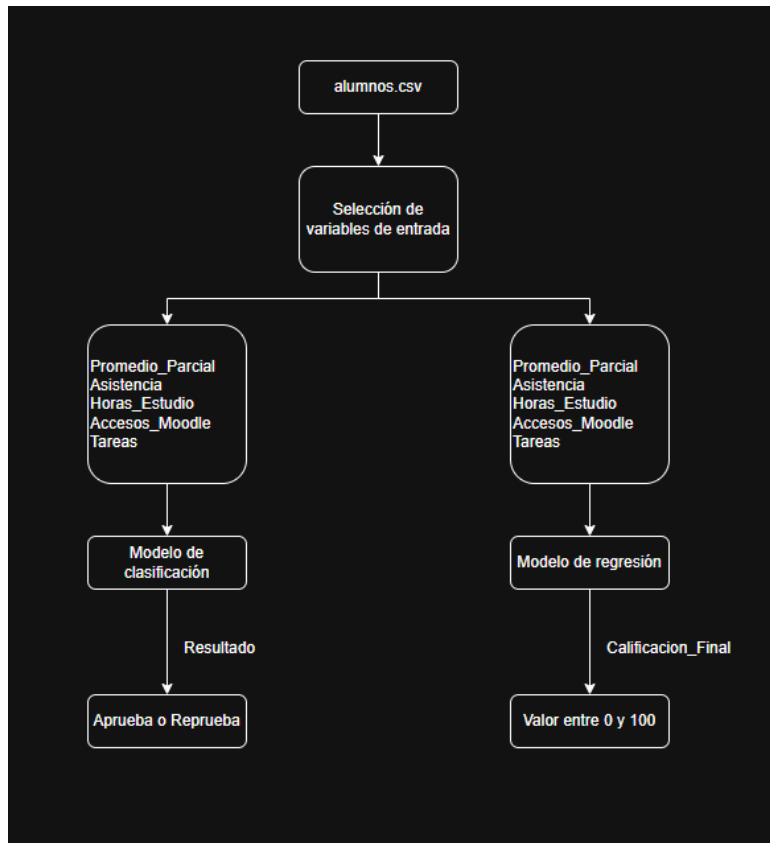
Evidencia de la Fase 2

Evidencia 1. Tabla de variables de entrada y objetivo

Modelo	Variable	Función	Tipo de dato
Clasificación	Promedio_Parcial	Entrada	Numérico entero
Clasificación	Asistencia	Entrada	Numérico entero
Clasificación	Horas_Estudio	Entrada	Numérico decimal
Clasificación	Accesos_Moodle	Entrada	Numérico entero
Clasificación	Tareas	Entrada	Numérico entero
Clasificación	Resultado	Objetivo	Categorico binario
Regresión	Promedio_Parcial	Entrada	Numérico entero
Regresión	Asistencia	Entrada	Numérico entero

Regresión	Horas_Estudio	Entrada	Numérico decimal
Regresión	Accesos_Moodle	Entrada	Numérico entero
Regresión	Tareas	Entrada	Numérico entero
Regresión	Calificacion_Final	Objetivo	Numérico

Evidencia 2. Diagrama del flujo de datos



Evidencia 3. Separación de variables en Google Colab

Se cargó el archivo alumnos.csv y se separaron correctamente las variables de entrada y las variables objetivo para los modelos de clasificación y regresión. Para clasificación se utilizó Resultado y para regresión Calificacion_Final, evitando fuga de información.

```
import pandas as pd

datos = pd.read_csv("/content/data/alumnos.csv")

datos.head()
```

	Promedio_Parcial	Asistencia	Horas_Estudio	Accesos_Moodle	Tareas	Calificacion_Final	Resultado
0	92	81	6.2	40	9	90	Aprueba
1	56	100	12.4	30	7	78	Aprueba
2	57	78	3.4	23	7	67	Reprueba
3	52	83	6.4	35	7	73	Aprueba
4	87	85	9.3	33	10	96	Aprueba

Próximos pasos: [Generar código con datos](#) [New interactive sheet](#)

```
variables_entrada = [
    "Promedio_Parcial",
    "Asistencia",
    "Horas_Estudio",
    "Accesos_Moodle",
    "Tareas"
]

X_clasificacion = datos[variables_entrada]
y_clasificacion = datos["Resultado"]

X_regresion = datos[variables_entrada]
y_regresion = datos["Calificacion_Final"]
```

```
(4)
✓ 0s
print("Entradas para clasificación:")
display(X_clasificacion.head())

print("Objetivo de clasificación:")
display(y_clasificacion.head())

print("Entradas para regresión:")
display(X_regresion.head())

print("Objetivo de regresión:")
display(y_regresion.head())
```

Entradas para clasificación:

	Promedio_Parcial	Asistencia	Horas_Estudio	Accesos_Moodle	Tareas
0	92	81	6.2	40	9
1	56	100	12.4	30	7
2	57	78	3.4	23	7
3	52	83	6.4	35	7
4	87	85	9.3	33	10

Objetivo de clasificación:

	Resultado
0	Aprueba
1	Aprueba
2	Reprueba
3	Aprueba
4	Aprueba

```
dtype: object
```

Entradas para regresión:

	Promedio_Parcial	Asistencia	Horas_Estudio	Accesos_Moodle	Tareas
0	92	81	6.2	40	9
1	56	100	12.4	30	7
2	57	78	3.4	23	7
3	52	83	6.4	35	7
4	87	85	9.3	33	10

Objetivo de regresión:

	Calificacion_Final
0	90
1	78
2	67
3	73
4	96

```
dtype: int64
```

Problemas encontrados y solución aplicada

Problema: Las variables Calificacion_Final y Resultado se encuentran directamente relacionadas, ya que el resultado se determina según si la calificación es mayor o igual a 70. Utilizar una de estas variables para predecir la otra habría provocado fuga de información y resultados artificialmente altos.

Solución: Se decidió excluir Calificacion_Final de las entradas del modelo de clasificación y excluir Resultado de las entradas del modelo de regresión. Ambos modelos utilizarán únicamente el promedio parcial, asistencia, horas de estudio, accesos a Moodle y tareas.

Resultado: Se definieron dos conjuntos de variables sin información directa sobre la respuesta final. Esto permitirá que los modelos aprendan patrones reales relacionados con el rendimiento académico y que las métricas obtenidas sean más confiables.

Conclusión de la fase

En esta fase se definieron claramente los dos problemas que serán desarrollados durante el proyecto. El modelo de clasificación buscará determinar si un estudiante aprobará o reprobará, mientras que el modelo de regresión estimará su calificación final.

También se establecieron las cinco variables de entrada que serán compartidas por ambos modelos. Estas variables representan el desempeño, la asistencia, los hábitos de estudio, la actividad en Moodle y el cumplimiento de tareas.

Finalmente, se tomaron medidas para evitar la fuga de información entre las variables objetivo. Con estas decisiones, el proyecto cuenta con una estructura clara para comenzar la selección, implementación y evaluación de los algoritmos.

Fase 3. Justificación de los algoritmos

Objetivo de la fase

Explicar por qué se seleccionó cada algoritmo candidato y qué se espera de su desempeño en los problemas de clasificación y regresión.

El documento del proyecto permite utilizar distintos algoritmos para ambos modelos y solicita justificar la elección de cada uno antes de su implementación.

Procedimiento

1. Clasificación

Justificar Regresión Logística, Árbol de Decisión y Random Forest.

Procedimiento realizado: Para el problema de clasificación se seleccionaron tres algoritmos candidatos: Regresión Logística, Árbol de Decisión y Random Forest. Cada uno presenta características diferentes, lo que permitirá comparar un modelo sencillo con otros capaces de representar relaciones más complejas.

Regresión Logística

La Regresión Logística se eligió como modelo inicial porque es adecuada para problemas de clasificación binaria, como predecir si un estudiante aprueba o reprueba.

Aunque su nombre contiene la palabra “regresión”, este algoritmo se utiliza para estimar la probabilidad de que un registro pertenezca a una categoría determinada.

Sus principales ventajas son:

- Es sencilla de implementar.
- Tiene un tiempo de entrenamiento rápido.
- Permite obtener una referencia inicial.
- Facilita la interpretación de la influencia de las variables.
- Funciona correctamente cuando la relación entre las entradas y la probabilidad de aprobar es relativamente simple.

Se espera que este modelo obtenga resultados aceptables y sirva como punto de comparación para los modelos más complejos.

Árbol de Decisión

El Árbol de Decisión se seleccionó porque permite clasificar a los estudiantes mediante reglas fáciles de interpretar.

Por ejemplo, el modelo podría aprender condiciones relacionadas con:

- El promedio parcial.
- El porcentaje de asistencia.

- La cantidad de tareas entregadas.
- Las horas de estudio.
- Los accesos a Moodle.

Una regla podría ser:

Si el promedio parcial es alto, la asistencia es suficiente y se entregó la mayoría de las tareas, aumenta la posibilidad de aprobar.

Sus principales ventajas son:

- Es sencillo de visualizar.
- Sus decisiones pueden explicarse mediante condiciones.
- Permite identificar las variables más importantes.
- Puede representar relaciones que no sean completamente lineales.

Se espera que este algoritmo pueda detectar patrones más complejos que la Regresión Logística. Sin embargo, deberá revisarse que no se ajuste demasiado a los datos de entrenamiento.

Random Forest

Random Forest se seleccionó porque combina varios árboles de decisión y utiliza el resultado conjunto para realizar la clasificación.

Al utilizar múltiples árboles, puede reducir algunos errores que podría cometer un árbol individual y generar predicciones más estables.

Sus principales ventajas son:

- Puede representar relaciones complejas.
- Generalmente ofrece resultados estables.
- Reduce el riesgo de depender de una sola estructura de decisión.
- Permite conocer la importancia de las variables.
- Puede manejar correctamente datos tabulares como los del proyecto.

Se espera que Random Forest obtenga uno de los mejores resultados de clasificación. No obstante, la selección final dependerá de las métricas obtenidas y no únicamente de que sea un modelo más avanzado.

2. Regresión

Justificar Regresión Lineal, Decision Tree Regressor y Random Forest Regressor.

Procedimiento realizado: Para el problema de regresión se seleccionaron Regresión Lineal, Decision Tree Regressor y Random Forest Regressor. Estos algoritmos permitirán comparar un modelo lineal sencillo con modelos basados en árboles.

Regresión Lineal

La Regresión Lineal se eligió como modelo inicial porque permite predecir un valor numérico y estudiar la relación entre las variables de entrada y la calificación final.

El algoritmo intentará identificar cómo influyen en la calificación variables como:

- Promedio parcial.
- Asistencia.
- Horas de estudio.
- Accesos a Moodle.
- Tareas.

Sus principales ventajas son:

- Es sencilla de implementar.
- Tiene un entrenamiento rápido.
- Sus resultados son fáciles de interpretar.
- Permite establecer una referencia inicial.
- Funciona correctamente cuando las relaciones entre las variables son aproximadamente lineales.

Se espera que este modelo obtenga un desempeño adecuado, debido a que el dataset fue generado con relaciones coherentes entre las variables académicas y la calificación final.

Decision Tree Regressor

Decision Tree Regressor se seleccionó porque puede realizar predicciones numéricas mediante reglas de decisión.

A diferencia de la Regresión Lineal, este modelo no necesita que todas las relaciones sean lineales. Puede dividir los datos en diferentes grupos según sus características.

Por ejemplo, podría aprender que:

- Los estudiantes con promedio parcial alto y muchas tareas suelen tener calificaciones finales elevadas.
- Los estudiantes con baja asistencia y pocas tareas suelen obtener calificaciones menores.
- Las horas de estudio pueden influir de forma diferente dependiendo del promedio parcial.

Sus principales ventajas son:

- Representa relaciones no lineales.
- Es relativamente fácil de interpretar.

- Puede mostrar la importancia de las variables.
- No requiere normalizar necesariamente los datos.

Se espera que este modelo pueda mejorar las predicciones en aquellos casos donde la relación entre las variables sea más compleja. También se deberá revisar el posible sobreajuste.

Random Forest Regressor

Random Forest Regressor se seleccionó porque combina las predicciones de varios árboles de regresión.

En lugar de depender de un solo árbol, genera múltiples modelos y calcula una predicción conjunta, normalmente mediante el promedio de sus resultados.

Sus principales ventajas son:

- Puede representar relaciones complejas.
- Suele ser más estable que un solo árbol.
- Reduce el riesgo de sobreajuste respecto a un árbol individual.
- Puede manejar varias variables de entrada.
- Permite analizar la importancia de las características.

Se espera que Random Forest Regressor obtenga uno de los mejores resultados para predecir la calificación final, especialmente si existen relaciones no lineales entre las variables.

3. Criterios de comparación

Describir las métricas que se usarán para seleccionar los modelos finales.

Procedimiento realizado: Describir las métricas que se usarán para seleccionar los modelos finales.

Procedimiento realizado:

La selección de los modelos finales se realizará mediante métricas específicas para clasificación y regresión.

Métricas para clasificación

Accuracy

Mide la proporción total de predicciones correctas.

Permitirá conocer qué porcentaje de estudiantes fue clasificado correctamente como aprobado o reprobado.

Precision

Mide qué tan correctas son las predicciones realizadas para una categoría determinada.

En este proyecto ayudará a analizar cuántos estudiantes identificados por el modelo pertenecen realmente a la clase predicha.

Recall

Mide la capacidad del modelo para identificar correctamente los registros que pertenecen a una categoría.

Será especialmente importante para evaluar qué tan bien detecta a los estudiantes que podrían reprobar.

F1-score

Combina Precision y Recall en una sola métrica.

Será útil para comparar los modelos cuando sea importante mantener un equilibrio entre ambas medidas.

Matriz de confusión

Permitirá observar:

- Aprobados identificados correctamente.
- Reprobados identificados correctamente.
- Aprobados clasificados como reprobados.
- Reprobados clasificados como aprobados.

La matriz de confusión ayudará a entender los tipos de errores que comete cada algoritmo.

Métricas para regresión

MAE

El error absoluto medio indicará la diferencia promedio entre las calificaciones reales y las predichas.

Un MAE menor representará un mejor desempeño.

MSE

El error cuadrático medio dará mayor importancia a las predicciones que se encuentren muy alejadas de los valores reales.

Un valor menor será preferible.

RMSE

La raíz del error cuadrático medio mostrará el error en la misma escala que las calificaciones.

Por ejemplo, un RMSE de 5 indicaría que las predicciones se alejan aproximadamente cinco puntos de las calificaciones reales.

R²

El coeficiente de determinación indicará qué proporción del comportamiento de la calificación final puede ser explicada por el modelo.

Un valor cercano a 1 representará un mejor ajuste.

Criterio final de selección

El mejor modelo no se elegirá utilizando una sola métrica. También se considerarán:

- La estabilidad de sus resultados.
- La diferencia entre entrenamiento y prueba.
- La presencia de sobreajuste.
- La facilidad de interpretación.
- El tipo de errores cometidos.
- La utilidad del modelo dentro del caso de estudio.

4. Modelo base

Identificar el modelo inicial que servirá como referencia para la optimización.

Procedimiento realizado: Se definieron dos modelos base, uno para cada tipo de problema.

Para clasificación se utilizará como modelo base:

Regresión Logística

Se eligió porque es sencilla, rápida y adecuada para una variable objetivo con dos categorías.

Para regresión se utilizará como modelo base:

Regresión Lineal

Se eligió porque permite predecir valores numéricos y establecer una referencia fácil de interpretar.

Los resultados de estos modelos se compararán posteriormente con:

- Árbol de Decisión.
- Random Forest.
- Decision Tree Regressor.
- Random Forest Regressor.

La optimización consistirá en comprobar si los modelos basados en árboles mejoran las métricas obtenidas por los modelos base.

Resultados obtenidos

Como resultado de esta fase se seleccionaron seis algoritmos candidatos.

Para clasificación se utilizarán:

- Regresión Logística.
- Árbol de Decisión.
- Random Forest.

Para regresión se utilizarán:

- Regresión Lineal.
- Decision Tree Regressor.
- Random Forest Regressor.

También se definieron las métricas con las que serán evaluados. En clasificación se utilizarán Accuracy, Precision, Recall, F1-score y matriz de confusión. En regresión se utilizarán MAE, MSE, RMSE y R^2 .

La Regresión Logística y la Regresión Lineal fueron seleccionadas como modelos base. Sus resultados servirán como referencia para comprobar si los modelos basados en árboles ofrecen una mejora real.

No se seleccionó todavía un algoritmo final, ya que esa decisión se realizará después del entrenamiento y evaluación de todos los candidatos.

Evidencia de la Fase 3

Evidencia 1. Tabla comparativa de algoritmos

Problema	Algoritmo	Característica principal	Ventaja	Posible limitación
Clasificación	Regresión Logística	Modelo binario sencillo	Rápido y fácil de interpretar	Puede tener dificultades con relaciones complejas
Clasificación	Árbol de Decisión	Utiliza reglas de decisión	Fácil de visualizar	Puede presentar sobreajuste
Clasificación	Random Forest	Combina varios árboles	Mayor estabilidad	Menor facilidad de interpretación
Regresión	Regresión Lineal	Predicción mediante relaciones lineales	Sencilla y rápida	Puede no representar relaciones no lineales
Regresión	Decision Tree Regressor	Predice mediante divisiones y reglas	Maneja relaciones complejas	Puede presentar sobreajuste

Regresión	Random Forest Regressor	Promedia varios árboles	Predicciones estables	Requiere mayor procesamiento
-----------	-------------------------	-------------------------	-----------------------	------------------------------

Problemas encontrados y solución aplicada

Problema: Los algoritmos seleccionados presentan diferentes niveles de complejidad y no era posible determinar cuál sería el mejor antes de entrenarlos. Además, elegir directamente el modelo más avanzado podría dificultar la justificación de la mejora obtenida.

Solución: Se decidió utilizar modelos sencillos como referencia inicial y compararlos posteriormente con modelos basados en árboles. La selección final se realizará utilizando las métricas correspondientes a cada problema y revisando también la estabilidad y el posible sobreajuste.

Resultado: Se estableció un proceso de comparación claro y ordenado. La Regresión Logística será el modelo base de clasificación y la Regresión Lineal será el modelo base de regresión. Los demás algoritmos permitirán evaluar si existe una mejora real en el desempeño.

Conclusión de la fase

En esta fase se justificaron los algoritmos candidatos que serán utilizados para desarrollar los modelos de clasificación y regresión.

La selección incluye modelos sencillos y modelos más complejos, lo que permitirá comparar sus resultados de manera clara. La Regresión Logística y la Regresión Lineal servirán como referencia inicial, mientras que los algoritmos basados en árboles serán utilizados como posibles mejoras.

También se establecieron las métricas de evaluación que permitirán seleccionar los modelos finales. Con estas decisiones, el proyecto se encuentra preparado para iniciar la implementación y el entrenamiento de los modelos.

Fase 4. Implementación de los modelos

Objetivo de la fase

Desarrollar los notebooks de clasificación y regresión en Google Colab, preparando los datos, entrenando los algoritmos candidatos y generando predicciones con el conjunto de prueba.

Procedimiento

1. Importación de librerías

Cargar Pandas, NumPy, Matplotlib y los componentes necesarios de Scikit-Learn.

Procedimiento realizado: Se importaron las librerías necesarias para cargar y procesar el dataset, dividir los datos, entrenar los modelos y generar predicciones.

Para la manipulación de datos se utilizaron Pandas y NumPy. Matplotlib se importó para la elaboración posterior de gráficas. También se cargaron los algoritmos de clasificación y regresión seleccionados, además de la función `train_test_split`, utilizada para separar los conjuntos de entrenamiento y prueba.

Código utilizado:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

2. Lectura del dataset

Cargar `alumnos.csv` y revisar su estructura.

Procedimiento realizado: Se cargó el archivo `alumnos.csv` mediante la función `read_csv()` de Pandas. Después se mostraron las primeras filas para comprobar que las columnas y los registros fueran reconocidos correctamente.

También se revisó la cantidad de filas y columnas, los tipos de datos y la presencia de valores vacíos.

Código utilizado:

```
datos = pd.read_csv("/content/data/alumnos.csv")

display(datos.head())

print("Dimensiones del dataset:", datos.shape)
print("\nTipos de datos:")
print(datos.dtypes)

print("\nValores vacíos:")
print(datos.isnull().sum())
```

El resultado esperado es:

200 filas

7 columnas

3. Preparación

Separar variables independientes y objetivo; transformar Resultado cuando sea necesario.

Procedimiento realizado: Se seleccionaron como variables independientes:

- Promedio_Parcial
- Asistencia
- Horas_Estudio
- Accesos_Moodle
- Tareas

Para el modelo de clasificación, la variable objetivo fue Resultado. Esta columna se transformó a valores numéricos para que los algoritmos pudieran procesarla:

Aprueba = 1

Reprueba = 0

Para el modelo de regresión, la variable objetivo fue Calificacion_Final.

Código utilizado:

```
variables_entrada = [  
    "Promedio_Parcial",  
    "Asistencia",  
    "Horas_Estudio",  
    "Accesos_Moodle",  
    "Tareas"  
]  
  
X_clasificacion = datos[variables_entrada]  
  
y_clasificacion = datos["Resultado"].map({  
    "Aprueba": 1,  
    "Reprueba": 0  
})
```



```

X_regresion = datos[variables_entrada]
y_regresion = datos["Calificacion_Final"]

Para comprobar la transformación:

print("Valores originales:")
print(datos["Resultado"].value_counts())

print("\nValores transformados:")
print(y_clasificacion.value_counts())

```

4. División de datos

Separar 80 % para entrenamiento y 20 % para prueba.

Procedimiento realizado: Los datos fueron divididos mediante `train_test_split`. Se utilizó el 80 % de los registros para entrenar los modelos y el 20 % restante para evaluar sus predicciones.

Se utilizó `random_state=42` para mantener la misma división cada vez que se ejecutara el notebook.

En clasificación se agregó `stratify=y_clasificacion` para conservar una proporción similar de estudiantes aprobados y reprobados en ambos conjuntos.

Código de clasificación:

```

X_train_clasificacion, X_test_clasificacion, y_train_clasificacion, y_test_clasificacion =
train_test_split(
    X_clasificacion,
    y_clasificacion,
    test_size=0.20,
    random_state=42,
    stratify=y_clasificacion
)

```

Código de regresión:

```

X_train_regresion, X_test_regresion, y_train_regresion, y_test_regresion = train_test_split(
    X_regresion,
    y_regresion,
    test_size=0.20,
    random_state=42
)

```

Comprobación:

```
print("Clasificación:")
print("Entrenamiento:", X_train_clasificacion.shape)
print("Prueba:", X_test_clasificacion.shape)
```

```
print("\nRegresión:")
print("Entrenamiento:", X_train_regresion.shape)
print("Prueba:", X_test_regresion.shape)
```

El resultado esperado es:

Entrenamiento: 160 registros

Prueba: 40 registros

5. Entrenamiento

Entrenar los algoritmos candidatos bajo condiciones comparables.

Procedimiento realizado: Entrenar los algoritmos candidatos bajo condiciones comparables.

Procedimiento realizado:

Se crearon y entrenaron tres algoritmos de clasificación y tres algoritmos de regresión. Todos fueron entrenados utilizando las mismas variables y la misma división de datos para permitir una comparación justa.

Modelos de clasificación

```
modelo_logistico = LogisticRegression(max_iter=1000)
modelo_arbol_clasificacion = DecisionTreeClassifier(random_state=42)
modelo_random_forest_clasificacion = RandomForestClassifier(
    random_state=42
)

modelo_logistico.fit(
    X_train_clasificacion,
    y_train_clasificacion
)

modelo_arbol_clasificacion.fit(
    X_train_clasificacion,
    y_train_clasificacion
)
```

```

modelo_random_forest_clasificacion.fit(
    X_train_clasificacion,
    y_train_clasificacion
)

```

Modelos de regresión

```

modelo_lineal = LinearRegression()
modelo_arbol_regresion = DecisionTreeRegressor(random_state=42)
modelo_random_forest_regresion = RandomForestRegressor(
    random_state=42
)

```

```

modelo_lineal.fit(
    X_train_regresion,
    y_train_regresion
)

```

```

modelo_arbol_regresion.fit(
    X_train_regresion,
    y_train_regresion
)

```

```

modelo_random_forest_regresion.fit(
    X_train_regresion,
    y_train_regresion
)

```

Al finalizar el entrenamiento, los seis modelos quedaron preparados para generar predicciones.

6. Predicción

Generar predicciones con el conjunto de prueba.

Procedimiento realizado: Se utilizaron los modelos entrenados para generar predicciones con los 40 registros reservados para prueba.

Predicciones de clasificación

```

pred_logistico = modelo_logistico.predict(
    X_test_clasificacion
)

```

```
)
```

```
pred_arbol_clasificacion = modelo_arbol_clasificacion.predict(
    X_test_clasificacion
```

```
)
```

```
pred_random_forest_clasificacion = modelo_random_forest_clasificacion.predict(
    X_test_clasificacion
```

```
)
```

Predicciones de regresión

```
pred_lineal = modelo_lineal.predict(
    X_test_regresion
```

```
)
```

```
pred_arbol_regresion = modelo_arbol_regresion.predict(
    X_test_regresion
```

```
)
```

```
pred_random_forest_regresion = modelo_random_forest_regresion.predict(
    X_test_regresion
```

```
)
```

Para observar algunos ejemplos de clasificación:

```
comparacion_clasificacion = pd.DataFrame({
    "Resultado_real": y_test_clasificacion.values,
    "Regresion_Logistica": pred_logistico,
    "Arbol_Decision": pred_arbol_clasificacion,
    "Random_Forest": pred_random_forest_clasificacion
})
```

```
display(comparacion_clasificacion.head(10))
```

Para observar algunos ejemplos de regresión:

```
comparacion_regresion = pd.DataFrame({
    "Calificacion_real": y_test_regresion.values,
```

```

"Regresion_Lineal": pred_lineal,
"Arbol_Decision": pred_arbol_regresion,
"Random_Forest": pred_random_forest_regresion
})

```

```
display(comparacion_regresion.head(10))
```

Resultados obtenidos

Como resultado de esta fase se desarrollaron los modelos iniciales de clasificación y regresión en Google Colab.

El dataset fue cargado correctamente y se confirmó que contiene 200 registros y siete columnas. Las variables independientes fueron separadas de las variables objetivo y la columna Resultado fue transformada a valores numéricos.

Los datos fueron divididos de la siguiente manera:

- 160 registros para entrenamiento.
- 40 registros para prueba.

Se entrenaron los siguientes modelos de clasificación:

- Regresión Logística.
- Árbol de Decisión.
- Random Forest.

También se entrenaron los siguientes modelos de regresión:

- Regresión Lineal.
- Decision Tree Regressor.
- Random Forest Regressor.

Finalmente, se generaron predicciones con los datos de prueba. Estas predicciones serán evaluadas en la siguiente fase mediante las métricas correspondientes.

Evidencias opcionales

- Captura de importación y lectura del CSV.
- Resumen del dataset.
- Código de división de datos.
- Código de entrenamiento.

- Ejemplos de predicciones.

Evidencia de la Fase 4

Evidencia 1. Carga y preparación del dataset

```
[ ] # 1. Importar librerías adicionales

[5]
✓ 2s import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

[ ] # Transformar Resultado

[6]
✓ 0s y_clasificacion_numerica = datos["Resultado"].map({
    "Aprueba": 1,
    "Reprueba": 0
})

print(y_clasificacion_numerica.value_counts())

Resultado
1    120
0     80
Name: count, dtype: int64
```

Evidencia 2. División y entrenamiento de los modelos

```
[ ] # Dividir los datos

[7]
✓ 0s X_train_clasificacion, X_test_clasificacion, y_train_clasificacion, y_test_clasificacion = train_test_split(
    X_clasificacion,
    y_clasificacion_numerica,
    test_size=0.20,
    random_state=42,
    stratify=y_clasificacion_numerica
)

X_train_regresion, X_test_regresion, y_train_regresion, y_test_regresion = train_test_split(
    X_regresion,
    y_regresion,
    test_size=0.20,
    random_state=42
)

print("Clasificación:")
print("Entrenamiento:", X_train_clasificacion.shape)
print("Prueba:", X_test_clasificacion.shape)

print("\nRegresión:")
print("Entrenamiento:", X_train_regresion.shape)
print("Prueba:", X_test_regresion.shape)

...
Clasificación:
Entrenamiento: (160, 5)
Prueba: (40, 5)

Regresión:
Entrenamiento: (160, 5)
Prueba: (40, 5)
```

```
[ ] # Entrenar clasificación

[8] ✓ 0s
modelo_logistico = LogisticRegression(max_iter=1000)

modelo_arbol_clasificacion = DecisionTreeClassifier(
    random_state=42
)

modelo_random_forest_clasificacion = RandomForestClassifier(
    random_state=42
)

modelo_logistico.fit(
    X_train_clasificacion,
    y_train_clasificacion
)

modelo_arbol_clasificacion.fit(
    X_train_clasificacion,
    y_train_clasificacion
)

modelo_random_forest_clasificacion.fit(
    X_train_clasificacion,
    y_train_clasificacion
)

print("Modelos de clasificación entrenados correctamente.")

Modelos de clasificación entrenados correctamente.
```

```
[ ] # Entrenar regresión

[9] ✓ 1s
modelo_lineal = LinearRegression()

modelo_arbol_regresion = DecisionTreeRegressor(
    random_state=42
)

modelo_random_forest_regresion = RandomForestRegressor(
    random_state=42
)

modelo_lineal.fit(
    X_train_regresion,
    y_train_regresion
)

modelo_arbol_regresion.fit(
    X_train_regresion,
    y_train_regresion
)

modelo_random_forest_regresion.fit(
    X_train_regresion,
    y_train_regresion
)

print("Modelos de regresión entrenados correctamente.")

Modelos de regresión entrenados correctamente.
```

Evidencia 3. Predicciones generadas

```
[11]
✓ 0 s
comparacion_clasificacion = pd.DataFrame({
    "Resultado_real": y_test_clasificacion.values,
    "Regresion_Logistica": pred_logistico,
    "Arbol_Decision": pred_arbol_clasificacion,
    "Random_Forest": pred_random_forest_clasificacion
})

display(comparacion_clasificacion.head(10))
```

	Resultado_real	Regresion_Logistica	Arbol_Decision	Random_Forest
0	0	0	0	0
1	1	1	1	1
2	1	1	0	0
3	1	1	1	1
4	0	0	0	0
5	0	0	0	0
6	1	1	1	1
7	1	1	1	1
8	0	0	0	0
9	0	0	0	0

```
[12]
✓ 0 s
comparacion_regresion = pd.DataFrame({
    "Calificacion_real": y_test_regresion.values,
    "Regresion_Lineal": pred_lineal.round(2),
    "Arbol_Decision": pred_arbol_regresion.round(2),
    "Random_Forest": pred_random_forest_regresion.round(2)
})

display(comparacion_regresion.head(10))
```

	Calificacion_real	Regresion_Lineal	Arbol_Decision	Random_Forest
0	44	41.26	36.0	40.35
1	81	77.51	80.0	73.21
2	88	87.35	92.0	89.89
3	64	61.13	58.0	60.95
4	81	82.70	79.0	82.36
5	40	39.35	36.0	39.71
6	98	97.67	94.0	94.94
7	89	91.30	84.0	88.39
8	71	68.24	71.0	66.70
9	60	68.96	69.0	67.23

Problemas encontrados y solución aplicada

Problema: La variable Resultado estaba almacenada como texto con los valores Aprueba y Reprueba, por lo que era necesario transformarla antes de utilizarla en los modelos de clasificación. Además, debía conservarse una proporción similar de ambas categorías en los datos de entrenamiento y prueba.

Solución: La variable fue convertida a valores numéricos, asignando 1 a los estudiantes aprobados y 0 a los reprobados. Durante la división de clasificación se utilizó el parámetro stratify, con el propósito de mantener la proporción de las dos clases.

Resultado: Los datos quedaron preparados correctamente, con 160 registros para entrenamiento y

40 para prueba. Los seis algoritmos candidatos fueron entrenados y pudieron generar predicciones sin utilizar información de las variables objetivo como entrada.

Conclusión de la fase

En esta fase se implementaron correctamente los modelos candidatos de clasificación y regresión en Google Colab.

El dataset fue cargado, revisado y preparado antes del entrenamiento. Las variables independientes y objetivo fueron separadas correctamente, y los registros se dividieron en conjuntos de entrenamiento y prueba utilizando una proporción de 80 % y 20 %.

Finalmente, se entrenaron seis algoritmos y se generaron sus primeras predicciones. Con ello, los modelos se encuentran preparados para calcular e interpretar sus métricas de evaluación en la siguiente fase.

Fase 5. Evaluación

Objetivo de la fase

Calcular e interpretar las métricas de los modelos de clasificación y regresión, con el propósito de comparar su desempeño e identificar qué algoritmo obtiene los mejores resultados.

Procedimiento

1. Clasificación

Calcular Accuracy, Precision, Recall, F1-score y matriz de confusión.

Procedimiento realizado: Se evaluaron los tres modelos de clasificación entrenados durante la fase anterior:

- Regresión Logística.
- Árbol de Decisión.
- Random Forest.

Para cada modelo se compararon las predicciones con los resultados reales del conjunto de prueba. Posteriormente, se calcularon las siguientes métricas:

- **Accuracy:** proporción total de estudiantes clasificados correctamente.
- **Precision:** proporción de predicciones positivas que fueron correctas.
- **Recall:** capacidad del modelo para identificar correctamente a los estudiantes de la clase analizada.
- **F1-score:** equilibrio entre Precision y Recall.
- **Matriz de confusión:** distribución de aciertos y errores según cada categoría.

Para facilitar la interpretación, se consideró:

0 = Reprueba

1 = Aprueba

La matriz de confusión permitió identificar cuatro tipos de resultados:

Resultado	Interpretación
Reprueba correctamente identificado	El modelo detectó correctamente a un estudiante que reprueba.
Aprueba correctamente identificado	El modelo detectó correctamente a un estudiante que aprueba.
Reprueba clasificado como Aprueba	El modelo no detectó a un estudiante en riesgo.
Aprueba clasificado como Reprueba	El modelo generó una alerta incorrecta.

En el contexto del proyecto, el error más importante será clasificar como aprobado a un estudiante que realmente reprobaría, ya que el sistema busca ayudar a los coordinadores antes de que ocurra la reprobación.

2. Regresión

Calcular MAE, MSE, RMSE y R^2 .

Procedimiento realizado: Se evaluaron los tres modelos de regresión entrenados:

- Regresión Lineal.
- Decision Tree Regressor.
- Random Forest Regressor.

Para cada algoritmo se compararon las calificaciones reales con las calificaciones predichas en los 40 registros del conjunto de prueba.

Se calcularon las siguientes métricas:

- **MAE:** diferencia absoluta promedio entre la calificación real y la predicha.
- **MSE:** promedio de los errores elevados al cuadrado.
- **RMSE:** raíz cuadrada del MSE, expresada en la misma escala de las calificaciones.
- **R^2 :** proporción del comportamiento de la calificación final explicada por el modelo.

En MAE, MSE y RMSE, los valores más bajos representan un mejor resultado. En R^2 , los valores más cercanos a 1 indican un mejor ajuste.

Por ejemplo, un MAE de 4 significaría que las predicciones se alejan aproximadamente cuatro puntos de la calificación real.

3. Interpretación

Explicar qué significan los valores obtenidos en el contexto académico.

Procedimiento realizado: Las métricas fueron interpretadas considerando el propósito del sistema universitario.

En clasificación, no solo se revisó el porcentaje total de aciertos. También se analizó la capacidad del modelo para identificar estudiantes que podrían reprobar. Un modelo con Accuracy elevada podría no ser suficiente si clasifica incorrectamente a varios estudiantes en riesgo.

Por esta razón, se consideraron conjuntamente:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Matriz de confusión.

En regresión, se analizó cuántos puntos se alejaron las predicciones de las calificaciones reales. El MAE y RMSE permitieron expresar el error en puntos de calificación, mientras que R^2 permitió conocer qué tanto del comportamiento de las notas fue explicado por el modelo.

La interpretación se realizó sobre todo el conjunto de prueba y no únicamente sobre las primeras predicciones mostradas en la fase anterior.

4. Comparación

Registrar los resultados de cada algoritmo en tablas.

Procedimiento realizado: Los resultados fueron organizados en dos tablas comparativas: una para clasificación y otra para regresión.

Para clasificación se registraron:

Algoritmo	Accuracy	Precision	Recall	F1-score
<i>Regresión Logística</i>	0.9250	0.9565	0.9167	0.9362
<i>Árbol de Decisión</i>	0.9000	1.0000	0.8333	0.9091
<i>Random Forest</i>	0.9000	0.9545	0.8750	0.9130

Para regresión se registraron:

Algoritmo	MAE	MSE	RMSE	R^2
<i>Regresión Lineal</i>	4.1919	31.8941	5.6475	0.9229
<i>Decision Tree Regressor</i>	5.8250	57.1750	7.5614	0.8617
<i>Random Forest Regressor</i>	5.3930	50.5266	7.1082	0.8778

La Regresión Logística obtuvo el mejor desempeño en clasificación, con un Accuracy de 92.5 % y un F1-score de 93.62 %. En regresión, la Regresión Lineal presentó los menores errores y el mayor valor de R^2 , por lo que fue el modelo con mejor desempeño.

Resultados obtenidos

Como resultado de la evaluación, la Regresión Logística presentó el mejor desempeño en clasificación, con un Accuracy de 92.5 % y un F1-score de 93.62 %. El modelo clasificó correctamente a 37 de los 40 estudiantes utilizados en la prueba.

En regresión, la Regresión Lineal obtuvo los mejores resultados, con un MAE de 4.19, un RMSE de 5.65 y un R^2 de 0.9229. Esto indica que sus predicciones se alejaron aproximadamente 4.19 puntos de las calificaciones reales y que logró explicar el 92.29 % de su comportamiento.

Por lo tanto, los mejores modelos iniciales fueron la Regresión Logística para clasificación y la Regresión Lineal para regresión.

Evidencia de la Fase 5

Evidencia 1. Comparación de métricas de clasificación

```
[14]
✓ 0 s

resultados_clasificacion = []

modelos_clasificacion = {
    "Regresión Logística": pred_logistico,
    "Árbol de Decisión": pred_arbol_clasificacion,
    "Random Forest": pred_random_forest_clasificacion
}

for nombre, predicciones in modelos_clasificacion.items():
    accuracy = accuracy_score(y_test_clasificacion, predicciones)
    precision = precision_score(y_test_clasificacion, predicciones)
    recall = recall_score(y_test_clasificacion, predicciones)
    f1 = f1_score(y_test_clasificacion, predicciones)

    resultados_clasificacion.append({
        "Algoritmo": nombre,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    })

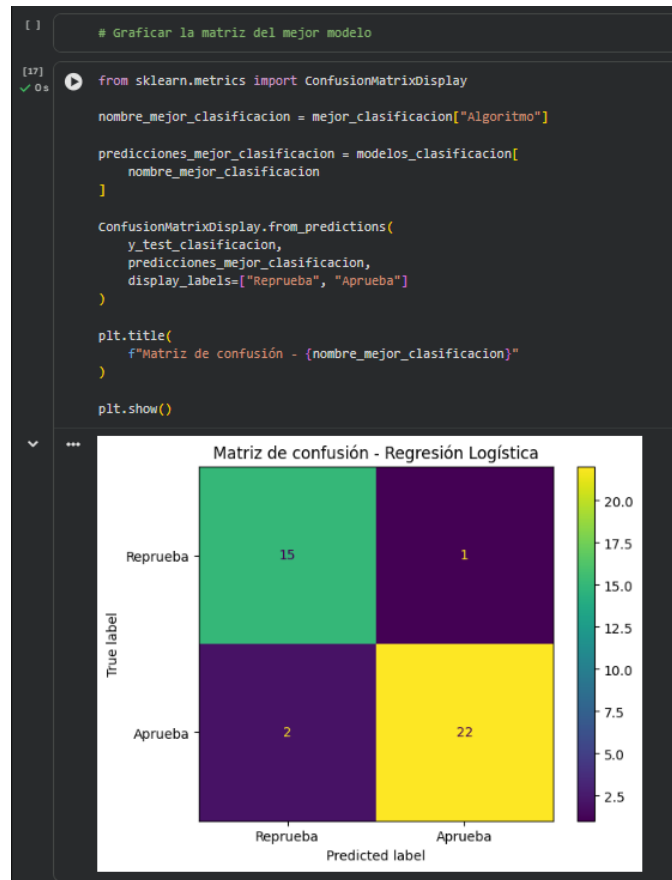
tabla_clasificacion = pd.DataFrame(resultados_clasificacion)

tabla_clasificacion[
    ["Accuracy", "Precision", "Recall", "F1-score"]
] = tabla_clasificacion[
    ["Accuracy", "Precision", "Recall", "F1-score"]
].round(4)

display(tabla_clasificacion)
```

	Algoritmo	Accuracy	Precision	Recall	F1-score
0	Regresión Logística	0.925	0.9565	0.9167	0.9362
1	Árbol de Decisión	0.900	1.0000	0.8333	0.9091
2	Random Forest	0.900	0.9545	0.8750	0.9130

Evidencia 2. Matriz de confusión del mejor modelo de clasificación



Evidencia 3. Comparación de métricas de regresión

```
[18]
0 s ▶ resultados_regresion = []

modelos_regresion = {
    "Regresión Lineal": pred_lineal,
    "Árbol de Decisión": pred_arbol_regresion,
    "Random Forest": pred_random_forest_regresion
}

for nombre, predicciones in modelos_regresion.items():
    mae = mean_absolute_error(
        y_test_regresion,
        predicciones
    )

    mse = mean_squared_error(
        y_test_regresion,
        predicciones
    )

    rmse = np.sqrt(mse)

    r2 = r2_score(
        y_test_regresion,
        predicciones
    )

    resultados_regresion.append({
        "Algoritmo": nombre,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R²": r2
    })

tabla_regresion = pd.DataFrame(resultados_regresion)

tabla_regresion[
    ["MAE", "MSE", "RMSE", "R²"]
] = tabla_regresion[
    ["MAE", "MSE", "RMSE", "R²"]
].round(4)

display(tabla_regresion)
```

	Algoritmo	MAE	MSE	RMSE	R²
0	Regresión Lineal	4.1919	31.8941	5.6475	0.9229
1	Árbol de Decisión	5.8250	57.1750	7.5614	0.8617
2	Random Forest	5.3930	50.5266	7.1082	0.8778

Problemas encontrados y solución aplicada

Problema: No era posible identificar el mejor algoritmo observando solamente algunos ejemplos de predicción, ya que el desempeño variaba entre los registros.

Solución: Se calcularon métricas especializadas para evaluar todas las predicciones del conjunto de prueba. En clasificación se utilizaron Accuracy, Precision, Recall, F1-score y matriz de confusión. En regresión se utilizaron MAE, MSE, RMSE y R^2 .

Resultado: La Regresión Logística fue seleccionada como el mejor modelo inicial de clasificación y la Regresión Lineal como el mejor modelo inicial de regresión.

Conclusión de la fase

La evaluación permitió comparar objetivamente los seis algoritmos implementados. La Regresión Logística obtuvo el mejor equilibrio entre las métricas de clasificación, mientras que la Regresión Lineal consiguió los menores errores y el mayor coeficiente de determinación en regresión.

Los resultados demuestran que un modelo más complejo no necesariamente produce mejores predicciones. En este dataset, los modelos base superaron a los algoritmos basados en árboles.

Estas métricas servirán como referencia para la siguiente fase, en la cual se intentará optimizar los modelos y comprobar si es posible mejorar su desempeño.

Fase 6. Optimización

Objetivo de la fase

Aplicar cambios en los modelos de clasificación y regresión para intentar mejorar su desempeño, comparar los resultados antes y después de la optimización y revisar si existe sobreajuste.

Procedimiento

1. Selección de estrategia

Cambiar algoritmo, ajustar hiperparámetros, normalizar o revisar variables.

Procedimiento realizado: Se aplicó normalización y ajuste de regularización a la Regresión Logística. Para los modelos basados en árboles se probaron diferentes valores de profundidad, número mínimo de muestras, criterio y cantidad de árboles. La búsqueda de parámetros se realizó mediante GridSearchCV con validación cruzada de cinco divisiones.

2. Reentrenamiento

Entrenar nuevamente con los cambios aplicados.

Procedimiento realizado: Los modelos fueron entrenados nuevamente utilizando los mejores parámetros encontrados durante la validación cruzada. Se conservaron los mismos conjuntos de entrenamiento y prueba para que la comparación con los modelos iniciales fuera justa.

3. Nueva evaluación

Calcular las mismas métricas para mantener una comparación justa.

Procedimiento realizado: Los modelos optimizados fueron evaluados mediante las mismas métricas utilizadas anteriormente. En clasificación se compararon Accuracy y F1-score. En regresión se compararon RMSE y R^2 .

La optimización no produjo mejoras en el conjunto de prueba. La Regresión Logística redujo su Accuracy de 0.9250 a 0.8250. El Árbol de Decisión y Random Forest mantuvieron sus resultados de clasificación.

En regresión, el Árbol de Decisión aumentó su RMSE de 7.5614 a 7.8778 y Random Forest aumentó de 7.1082 a 7.3552. La Regresión Lineal conservó el mejor resultado con un RMSE de 5.6475 y un R^2 de 0.9229.

4. Sobreajuste

Comparar entrenamiento y prueba para detectar si el modelo memoriza los datos.

Procedimiento realizado: Se compararon las métricas obtenidas en los datos de entrenamiento y prueba. Los modelos basados en árboles mostraron un desempeño mayor en entrenamiento que en prueba, lo que indica una posible tendencia al sobreajuste.

En clasificación, Random Forest tuvo la menor diferencia entre entrenamiento y prueba, con 0.0687. En regresión, la Regresión Lineal presentó el comportamiento más estable, mientras que los árboles mostraron diferencias cercanas a 0.08 en R^2 .

5. Conclusión de mejora

Determinar qué cambio produjo la mejora y por qué.

Procedimiento realizado: Se determinó que los cambios aplicados no produjeron una mejora real en los modelos. La validación cruzada encontró configuraciones adecuadas dentro del conjunto de entrenamiento, pero estas no superaron los resultados iniciales al utilizar los datos de prueba.

Por esta razón, se decidió conservar como modelos finales iniciales:

- Regresión Logística sin optimización para clasificación.
- Regresión Lineal para regresión.

Resultados obtenidos

La optimización permitió comprobar que ajustar hiperparámetros no garantiza una mejora en datos no observados.

En clasificación, la Regresión Logística optimizada redujo su Accuracy y F1-score. El Árbol de Decisión y Random Forest no presentaron cambios en sus métricas de prueba.

En regresión, los modelos basados en árboles aumentaron su error y disminuyeron su valor de R^2 . La Regresión Lineal continuó siendo el modelo con mejor desempeño y mayor estabilidad.

Evidencia de la Fase 6

Evidencia 1. Mejores hiperparámetros encontrados

```
*** Mejores parámetros de Regresión Logística:
{'modelo_C': 0.1, 'modelo_solver': 'lbfgs'}
Mejor F1 en validación cruzada:
0.9291
```

```
Mejores parámetros del Árbol de Decisión:
{'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}
Mejor F1 en validación cruzada:
0.915
```

```
Mejores parámetros de Random Forest:
{'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 100}
Mejor F1 en validación cruzada:
0.9334
```

```
Mejores parámetros del Árbol de Regresión:
{'criterion': 'squared_error', 'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2}
Mejor RMSE en validación cruzada:
6.8721
```

```
Mejores parámetros de Random Forest Regressor:
{'max_depth': 5, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 100}
Mejor RMSE en validación cruzada:
5.7697
```

Evidencia 2. Comparación antes y después

***	Modelo	Accuracy antes	Accuracy después	F1 antes	F1 después	
0	Regresión Logística	0.925	0.825	0.9362	0.8444	
1	Árbol de Decisión	0.900	0.900	0.9091	0.9091	
2	Random Forest	0.900	0.900	0.9130	0.9130	

***	Modelo	RMSE antes	RMSE después	R ² antes	R ² después	
0	Regresión Lineal	5.6475	5.6475	0.9229	0.9229	
1	Árbol de Decisión	7.5614	7.8778	0.8617	0.8499	
2	Random Forest	7.1082	7.3552	0.8778	0.8692	

Evidencia 3. Análisis de sobreajuste

***	Modelo	Accuracy entrenamiento	Accuracy prueba	Diferencia
0	Regresión Logística	0.9125	0.825	0.0875
1	Árbol de Decisión	0.9875	0.900	0.0875
2	Random Forest	0.9688	0.900	0.0687

***	Modelo	R ² entrenamiento	R ² prueba	Diferencia
0	Regresión Lineal	0.9026	0.9229	-0.0203
1	Árbol de Decisión optimizado	0.9336	0.8499	0.0837
2	Random Forest optimizado	0.9480	0.8692	0.0788

Problemas encontrados y solución aplicada

Problema: Los parámetros seleccionados mediante validación cruzada no mejoraron el desempeño de los modelos en el conjunto de prueba y algunos algoritmos mostraron señales de sobreajuste.

Solución: Se compararon directamente los resultados antes y después de la optimización, además de revisar las métricas de entrenamiento y prueba. Se decidió no reemplazar los modelos iniciales cuando los nuevos parámetros producían resultados inferiores.

Resultado: Se conservaron la Regresión Logística original como modelo final de clasificación y la Regresión Lineal como modelo final de regresión.

Conclusión de la fase

La optimización permitió evaluar diferentes configuraciones y analizar la estabilidad de los modelos. Sin embargo, los cambios no mejoraron las métricas obtenidas en el conjunto de prueba.

Los modelos basados en árboles mostraron una mayor diferencia entre entrenamiento y prueba, lo que indica tendencia al sobreajuste. En cambio, la Regresión Logística original y la Regresión Lineal mantuvieron los mejores resultados.

Por lo tanto, se concluyó que los modelos más sencillos fueron los más adecuados para este dataset. La optimización no fue inútil: demostró con resultados medibles que agregar complejidad no siempre mejora un modelo.

Fase 7. Resultados

Objetivo de la fase

Seleccionar los mejores modelos, analizar las variables que tuvieron mayor influencia en las predicciones y explicar cómo podrían utilizarse los resultados para detectar estudiantes en riesgo académico.

Procedimiento

1. Mejor algoritmo

Identificar el mejor modelo para clasificación y el mejor para regresión.

Procedimiento realizado: Se revisaron las métricas obtenidas durante la evaluación y la optimización de los modelos. Para clasificación se seleccionó la **Regresión Logística**, debido a que presentó el mejor equilibrio entre Accuracy, Precision, Recall y F1-score.

Para regresión se seleccionó la **Regresión Lineal**, ya que obtuvo los errores más bajos y el mayor valor de R^2 entre los algoritmos evaluados.

Los modelos finales fueron:

Problema	Modelo seleccionado	Métrica principal	Resultado	Métrica adicional	Resultado
Clasificación	Regresión Logística	F1-score	0.9362	Accuracy	0.9250
Regresión	Regresión Lineal	RMSE	5.6475	R^2	0.9229

2. Razones

Explicar por qué obtuvieron mejor desempeño.

Procedimiento realizado: La Regresión Logística obtuvo el mejor desempeño en clasificación porque logró un equilibrio adecuado entre los aciertos generales y la identificación correcta de estudiantes aprobados y reprobados. Su Accuracy de 0.9250 indica que clasificó correctamente al 92.5 % de los estudiantes del conjunto de prueba, mientras que su F1-score de 0.9362 muestra un buen equilibrio entre Precision y Recall.

La Regresión Lineal obtuvo el mejor desempeño en regresión porque presentó el menor error entre las calificaciones reales y las predichas. Su RMSE de 5.6475 indica que las predicciones se desviaron aproximadamente 5.65 puntos al considerar también los errores más grandes. Además, su R^2 de 0.9229 indica que el modelo explicó aproximadamente el 92.29 % de la variación de las calificaciones finales.

Los modelos más complejos no superaron a los modelos base. Esto se debe a que las relaciones presentes en el dataset pudieron ser representadas adecuadamente mediante modelos sencillos, mientras que los modelos basados en árboles mostraron una mayor tendencia al sobreajuste.

3. Variables importantes

Analizar qué variables influyeron más en las predicciones.

Procedimiento realizado: La importancia de las variables fue analizada mediante importancia por permutación. Este procedimiento modifica temporalmente los valores de cada variable y mide cuánto disminuye el desempeño del modelo.

En clasificación, las variables se ordenaron de la siguiente manera:

1. Promedio_Parcial
2. Tareas
3. Asistencia
4. Horas_Estudio
5. Accesos_Moodle

En regresión se obtuvo el mismo orden de importancia:

1. Promedio_Parcial
2. Tareas
3. Asistencia
4. Horas_Estudio
5. Accesos_Moodle

El Promedio_Parcial fue la variable con mayor influencia en ambos modelos. Esto indica que el desempeño acumulado del estudiante antes de la evaluación final es el principal indicador de su resultado académico y de su calificación final.

La cantidad de tareas entregadas y la asistencia también tuvieron una influencia considerable. Las horas de estudio y los accesos a Moodle aportaron información adicional, aunque su efecto fue menor.

4. Aprendizajes

Describir los conocimientos y dificultades encontradas.

Procedimiento realizado: Durante el desarrollo del proyecto se aprendió a construir y validar un dataset, separar variables de entrada y objetivo, dividir los datos en conjuntos de entrenamiento y prueba, entrenar diferentes algoritmos y comparar sus resultados mediante métricas especializadas.

También se comprendió la importancia de evitar la fuga de información, ya que utilizar la calificación final para predecir el resultado habría generado métricas artificialmente elevadas.

Una de las principales dificultades fue identificar el mejor modelo observando únicamente algunas predicciones. Esta dificultad se resolvió mediante el uso de métricas aplicadas a todo el conjunto de prueba.

Otro aprendizaje importante fue que la optimización no siempre mejora los resultados. Aunque se ajustaron hiperparámetros mediante validación cruzada, algunos modelos obtuvieron métricas inferiores en el conjunto de prueba. Esto demostró que un modelo más complejo no necesariamente es más adecuado.

5. Aplicación

Explicar cómo podrían utilizarse los modelos para detectar estudiantes en riesgo.

Procedimiento realizado: El modelo de clasificación puede utilizarse para estimar la probabilidad de que un estudiante repruebe a partir de su promedio parcial, asistencia, horas de estudio, accesos a Moodle y tareas entregadas.

Los estudiantes con una probabilidad alta de reprobación podrían ser incluidos en una lista de atención prioritaria. Los coordinadores académicos podrían utilizar esta información para:

- Contactar al estudiante.
- Revisar sus dificultades académicas.
- Recomendar asesorías.
- Dar seguimiento a sus tareas.
- Analizar su asistencia.
- Diseñar estrategias de recuperación.

En las predicciones de ejemplo se identificaron estudiantes con probabilidades de reprobación cercanas al 100 %. Estos casos presentaron, en general, promedios parciales bajos, pocas tareas entregadas, pocas horas de estudio o escasa actividad en Moodle.

El modelo de regresión también podría utilizarse para estimar la calificación final. Esto permitiría detectar estudiantes cuya calificación proyectada se encuentre cerca o por debajo de 70, incluso antes de finalizar el periodo académico.

Los modelos no deberían tomar decisiones definitivas por sí solos. Sus resultados funcionarían como una herramienta de apoyo para que los coordinadores identifiquen casos que requieren revisión.

Resultados obtenidos

La Regresión Logística fue seleccionada como el mejor modelo de clasificación, con un Accuracy de 92.5 % y un F1-score de 93.62 %.

La Regresión Lineal fue seleccionada como el mejor modelo de regresión, con un RMSE de 5.6475 y un R^2 de 0.9229. Esto indica que sus predicciones tuvieron un error aproximado de 5.65 puntos y que explicó el 92.29 % de la variación de las calificaciones finales.

El análisis de importancia mostró que el Promedio_Parcial fue la variable más relevante en ambos modelos. Después se ubicaron Tareas, Asistencia, Horas_Estudio y Accesos_Moodle.

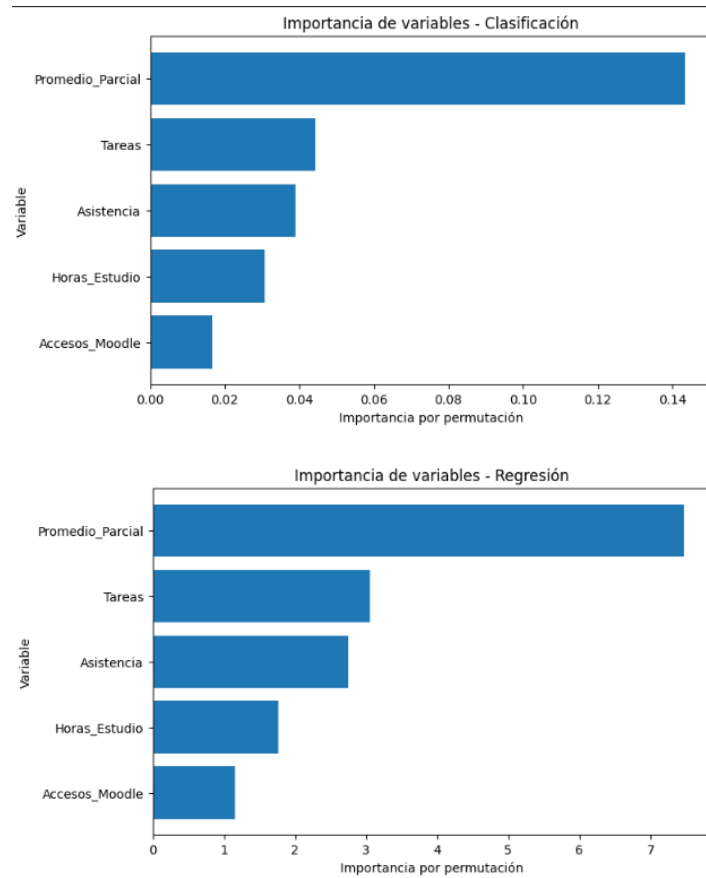
También se generaron predicciones de ejemplo para identificar estudiantes con mayor probabilidad de reprobación. Esto permitió demostrar cómo los modelos podrían utilizarse como apoyo para la intervención académica temprana.

Evidencia de la Fase 7

Evidencia 1. Modelos finales seleccionados

	Problema	Modelo seleccionado	Métrica principal	Resultado	Métrica adicional	Resultado adicional
0	Clasificación	Regresión Logística	F1-score	0.9362	Accuracy	0.9250
1	Regresión	Regresión Lineal	RMSE	5.6475	R ²	0.9229

Evidencia 2. Importancia de las variables



Evidencia 3. Estudiantes con mayor riesgo académico

***	Promedio_Parcial	Asistencia	Horas_Estudio	Accesos_Moodle	Tareas	Prediccion	Probabilidad_Reprobar
150	26	53	0.0	2	0	Reprueba	100.00
16	51	75	0.3	0	0	Reprueba	100.00
43	37	64	0.0	4	4	Reprueba	100.00
62	23	51	0.0	0	2	Reprueba	100.00
182	32	62	0.0	0	4	Reprueba	100.00
6	52	62	3.1	17	2	Reprueba	99.97
148	52	74	2.6	7	3	Reprueba	99.91
99	50	67	3.0	30	3	Reprueba	99.85
58	45	84	3.0	21	4	Reprueba	99.56
59	47	78	6.1	2	6	Reprueba	99.22

Problemas encontrados y solución aplicada

Problema: No era suficiente seleccionar los modelos únicamente por sus resultados generales, ya que también era necesario explicar qué variables influyeron en las predicciones y cómo podrían aplicarse los modelos en un contexto académico.

Solución: Se utilizaron las métricas finales para seleccionar los mejores modelos y se aplicó importancia por permutación para analizar la influencia de cada variable. Además, se generaron probabilidades de reprobación y predicciones de calificación para representar un posible uso del sistema.

Resultado: Se identificó que la Regresión Logística y la Regresión Lineal fueron los modelos más adecuados. También se determinó que el promedio parcial fue la variable con mayor influencia y se generó una lista de estudiantes con mayor riesgo académico.

Conclusión de la fase

La evaluación final permitió seleccionar la Regresión Logística como modelo de clasificación y la Regresión Lineal como modelo de regresión.

El promedio parcial fue la variable más importante, seguido por las tareas entregadas y la asistencia. Estos resultados muestran que el desempeño previo y el cumplimiento académico tienen una relación considerable con el resultado final del estudiante.

Los modelos podrían utilizarse como herramientas de apoyo para detectar estudiantes en riesgo y realizar intervenciones tempranas. Sin embargo, sus predicciones deben complementarse con la revisión y experiencia de los coordinadores académicos.

Fase 8. Publicación en GitHub

Objetivo de la fase

Organizar y publicar el proyecto con todos los archivos solicitados, verificando que el repositorio tenga una estructura clara, que los notebooks puedan ejecutarse correctamente y que la documentación sea accesible.

Procedimiento

1. Carga de archivos

Subir CSV, notebooks, imágenes, reporte y README.

Procedimiento realizado: Se creó y organizó el repositorio del proyecto en GitHub. Posteriormente, se subieron los archivos correspondientes al dataset, los notebooks de clasificación y regresión, las gráficas generadas durante el desarrollo y el archivo README.md.

También se agregó la carpeta Notebook_Completo, la cual contiene el archivo analisis_problema.ipynb. Este notebook conserva todo el desarrollo realizado desde la separación de variables hasta la evaluación, optimización y presentación de resultados finales.

La estructura utilizada fue la siguiente:

Proyecto_MachineLearning/

```
├── Dataset/
|   └── alumnos.csv
├── Clasificacion/
|   └── clasificacion.ipynb
├── Regresion/
|   └── regresion.ipynb
├── Notebook_Completo/
|   └── analisis_problema.ipynb
├── Imagenes/
|   ├── matriz_confusion_clasificacion.png
|   ├── valores_reales_predichos_regresion.png
|   ├── importancia_variables_clasificacion.png
|   └── importancia_variables_regresion.png
├── Documentacion/
|   └── Reporte.pdf
└── README.md
```

2. README

Incluir descripción, herramientas, integrantes, instrucciones y resultados.

Procedimiento realizado: Se elaboró el archivo README.md en la raíz del repositorio. Este documento contiene una descripción general del proyecto, los objetivos de clasificación y regresión, los integrantes del equipo, las herramientas utilizadas y la estructura de archivos.

También se incluyeron instrucciones para ejecutar los notebooks y cargar el archivo alumnos.csv. Finalmente, se registraron los principales resultados obtenidos:

- Regresión Logística como mejor modelo de clasificación.
- Regresión Lineal como mejor modelo de regresión.
- Accuracy de clasificación de 0.9250.
- F1-score de clasificación de 0.9362.
- RMSE de regresión de 5.6475.
- R^2 de regresión de 0.9229.

3. Revisión de ejecución

Comprobar que los notebooks se abren y ejecutan correctamente.

Procedimiento realizado: Se revisaron los notebooks `clasificacion.ipynb`, `regresion.ipynb` y `analisis_problema.ipynb`.

Cada notebook fue abierto en Google Colab y se comprobó que las celdas pudieran ejecutarse en orden. También se verificó que el usuario pudiera seleccionar y cargar manualmente el archivo `alumnos.csv` en la carpeta `data`, evitando depender de una ruta fija del equipo original.

Durante la revisión se comprobó que:

- Las librerías se importaran correctamente.
- El dataset pudiera cargarse.
- Las variables de entrada y objetivo se separaran correctamente.
- Los modelos pudieran entrenarse.
- Las predicciones y métricas se generaran sin errores.
- Las gráficas y tablas pudieran visualizarse.

4. Verificación final

Confirmar que la estructura coincide con la solicitada y que el repositorio es accesible.

Procedimiento realizado: Se realizó una revisión final de las carpetas y archivos del repositorio para comprobar que la estructura coincidiera con la solicitada en el proyecto.

Se verificó la existencia de:

- `Dataset/alumnos.csv`
- `Clasificacion/clasificacion.ipynb`
- `Regresion/regresion.ipynb`
- `Notebook_Completo/analisis_problema.ipynb`
- Carpeta `Imagenes`
- Carpeta `Documentacion`
- Archivo `README.md`

También se comprobó que los archivos pudieran abrirse desde GitHub y que el repositorio fuera accesible mediante su enlace. La carpeta `Notebook_Completo` se agregó como contenido adicional y no reemplaza los notebooks obligatorios de clasificación y regresión.

Resultados obtenidos

Como resultado de esta fase se obtuvo un repositorio organizado y accesible que contiene el dataset, los notebooks, las gráficas, la documentación y las instrucciones necesarias para comprender y ejecutar el proyecto.

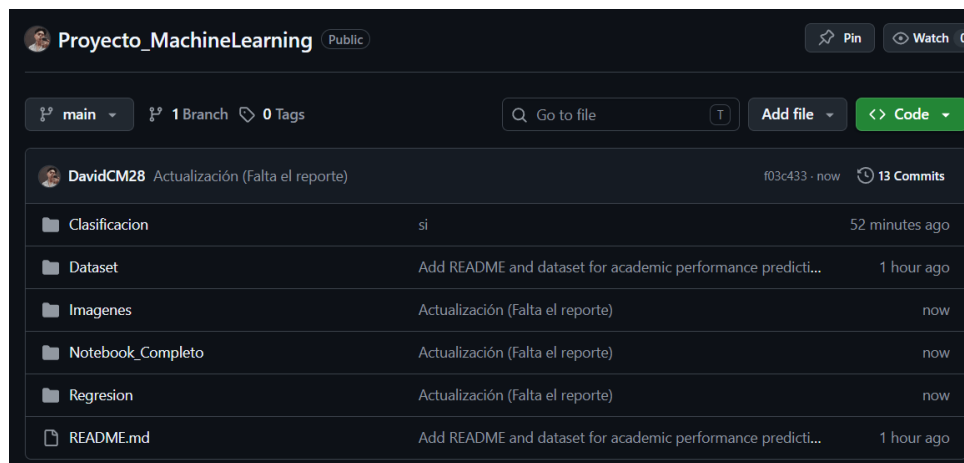
Los notebooks de clasificación y regresión fueron separados correctamente para cumplir con la estructura solicitada. Además, se conservó el notebook original completo como respaldo del desarrollo general.

El archivo README.md permite conocer rápidamente el objetivo del proyecto, los algoritmos utilizados, los resultados principales y el procedimiento necesario para ejecutar los modelos.

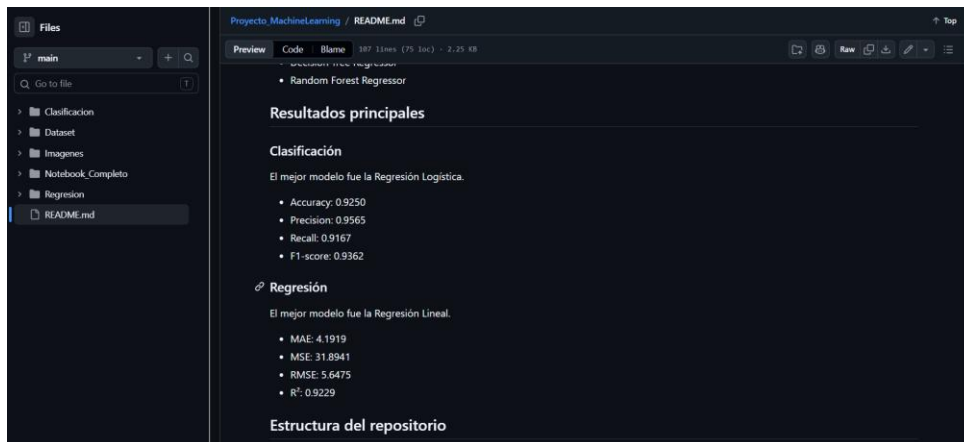
Una vez agregado el archivo Reporte.pdf, el repositorio quedará completamente preparado para su entrega final.

Evidencia de la Fase 8

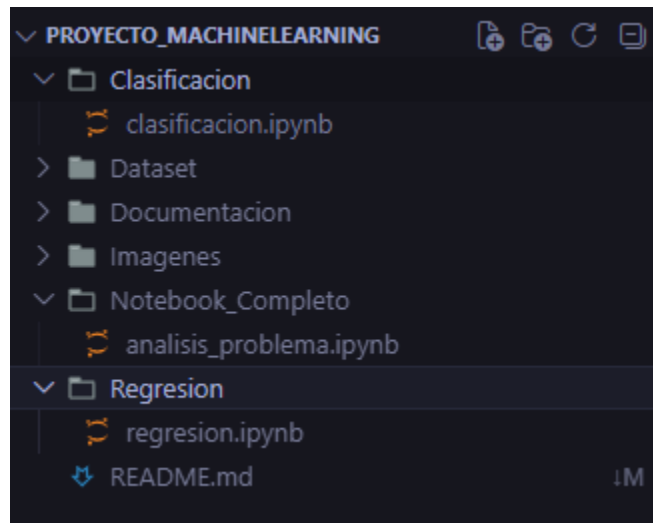
Evidencia 1. Estructura final del repositorio



Evidencia 2. Contenido del README



Evidencia 3. Notebooks publicados



Problemas encontrados y solución aplicada

Problema: El desarrollo completo se encontraba inicialmente en un solo notebook, lo que no coincidía con la estructura solicitada de tener un archivo independiente para clasificación y otro para regresión. Además, las rutas originales del dataset dependían del entorno de Google Colab.

Solución: Se crearon dos copias del notebook original y se separó el código según el tipo de problema. El notebook completo se conservó dentro de una carpeta adicional como respaldo.

Resultado: Se obtuvieron dos notebooks independientes y funcionales, además de un notebook general con todo el desarrollo. La estructura final cumple con los requisitos del proyecto y facilita la revisión y ejecución de los modelos.

Conclusión de la fase

La publicación en GitHub permitió reunir todos los elementos del proyecto en un repositorio organizado y accesible.

La separación de los notebooks facilitó la revisión individual de los modelos de clasificación y regresión, mientras que el notebook completo conserva el proceso general como referencia adicional.

Con la carga final del archivo Reporte.pdf, el proyecto quedará completamente documentado y listo para su entrega.

Conclusiones generales

Esta sección se completará al finalizar las ocho fases. Deberá sintetizar el desempeño de los modelos, las mejoras logradas, las limitaciones del dataset y los aprendizajes del equipo.

El proyecto permitió desarrollar un proceso completo de Machine Learning supervisado para analizar el rendimiento académico de estudiantes. Se construyó un dataset con 200 registros y se

implementaron modelos de clasificación para predecir si un estudiante aprobaría o reprobaría, así como modelos de regresión para estimar su calificación final.

En clasificación, la Regresión Logística obtuvo el mejor desempeño, con un Accuracy de 92.5 % y un F1-score de 93.62 %. En regresión, la Regresión Lineal presentó los mejores resultados, con un MAE de 4.19, un RMSE de 5.65 y un R^2 de 0.9229. Estos resultados demostraron que los modelos más sencillos fueron más adecuados para las relaciones presentes en el dataset.

Durante la optimización se probaron diferentes hiperparámetros y estrategias de normalización. Sin embargo, los modelos optimizados no superaron a los modelos iniciales en el conjunto de prueba. Este resultado permitió comprobar que aumentar la complejidad de un modelo no garantiza una mejora y que es necesario comparar objetivamente sus métricas antes de seleccionarlo.

El análisis de importancia mostró que el promedio parcial fue la variable con mayor influencia en ambos problemas, seguida por las tareas entregadas y la asistencia. Las horas de estudio y los accesos a Moodle también aportaron información, aunque tuvieron una influencia menor.

Una de las principales limitaciones del proyecto es que el dataset fue generado para fines académicos y contiene únicamente 200 registros. Por esta razón, los resultados no deben considerarse suficientes para implementar directamente el sistema en una universidad real. Para una aplicación futura sería necesario utilizar datos históricos reales, incluir más estudiantes y considerar otras variables académicas y personales.

Como aprendizaje general, el equipo comprendió la importancia de preparar correctamente los datos, evitar la fuga de información, separar entrenamiento y prueba, utilizar métricas adecuadas y analizar el sobreajuste. También se comprobó que los modelos pueden funcionar como una herramienta de apoyo para identificar estudiantes en riesgo, pero sus predicciones deben complementarse con la revisión de los coordinadores académicos.

Bibliografía

Registrar las fuentes consultadas para explicar los algoritmos, las métricas y las herramientas utilizadas. Utilizar un formato de citación consistente.

1. Scikit-learn Developers. (2026). Linear models: Logistic regression and linear regression. Scikit-learn documentation. Consultado el 27 de julio de 2026.
2. Scikit-learn Developers. (2026). Metrics and scoring: Quantifying the quality of predictions. Scikit-learn documentation. Consultado el 27 de julio de 2026.
3. Scikit-learn Developers. (2026). Decision tree and random forest models for classification and regression. Scikit-learn documentation. Consultado el 27 de julio de 2026.
4. The pandas Development Team. (2026). pandas.read_csv: Read a comma-separated values file into DataFrame. Pandas documentation. Consultado el 27 de julio de 2026.
5. Matplotlib Development Team. (2026). Pyplot tutorial. Matplotlib documentation. Consultado el 27 de julio de 2026.